Manual Técnico — Sistema Asilo Cabeza de Algodón

Oct 28, 2026 · @Fredy Vásquez

1. Resumen técnico

El sistema es un **monolito modular MVC en PHP 8.2** con una interfaz de una sola página (SPA) y **2 microservicios en Python/Flask**, sobre MariaDB. Son unas 9,900 líneas de código propio, sin frameworks de backend, y se publica con Docker en una máquina virtual de Azure.

Capa	Tecnología	Versión
Interfaz	HTML5, CSS3, JavaScript (ES2020), Bootstrap, Bootstrap Icons	Bootstrap 5.3.3
Servidor de aplicación	PHP con Apache	PHP 8.2
Base de datos	MariaDB / MySQL (InnoDB, utf8mb4)	MariaDB 10.11 (XAMPP: 10.4+)
Microservicios	Python, Flask, gunicorn	Python 3.10
Correo	PHPMailer (SMTP de Gmail) o Google Apps Script (HTTPS)	PHPMailer 6.9.3
Contenedores	Docker y Docker Compose	Compose v2
Proxy y HTTPS	Caddy con Let's Encrypt	Caddy 2
Nube	Azure, máquina virtual Ubuntu 24.04 ARM64	Standard B2pls v2

Decisiones de diseño

- **Monolito modular + 2 microservicios.** Los cálculos aislables (IMC y cotización) son microservicios; el resto comparte transacciones en una sola base, así que se mantiene en un monolito ordenado por módulos.

- **Front Controller único.** Toda petición de datos entra por `public/api.php`, que aplica en un solo lugar sesión, roles, CSRF y manejo de errores.
- **Tolerancia a fallas.** Si un microservicio no responde en 3 segundos, `ReglasClinicas` calcula con las mismas fórmulas en PHP.
- **Base de datos como fuente de verdad.** Reglas críticas (existencias, saldos, cuotas únicas) se protegen con transacciones, bloqueo `FOR UPDATE` y llaves foráneas.
- **Sin dependencias externas en tiempo de ejecución.** Bootstrap, íconos y fuentes se sirven desde el propio servidor, por lo que funciona sin Internet en XAMPP.

2. Estructura del código

El código sigue el patrón **MVC**: las vistas están en `public/`, los controladores en `src/Controllers/` y los modelos en `src/Models/`. Solo la carpeta `public/` es visible desde la web; la configuración, el código y la base de datos quedan fuera de su alcance.

```
asilo-cabeza-algodon/
├── config/            config.php (parámetros y roles), database.php
(conexión PDO Singleton), correo.ejemplo.php
├── database/          schema.sql: 25 tablas + datos de ejemplo
├── deploy/            azure/ (instalar.sh, correo.sh, reiniciar-datos.sh) y
caddy/Caddyfile
├── docker/            ms-imc/ y ms-cotizador/ (Flask), web/Dockerfile (PHP +
Apache)
├── public/            Raíz web
|   ├── api.php            Front Controller: tabla de 68 rutas
|   ├── login.php          Ingreso y ¿Olvidó su contraseña?
|   ├── index.php          Panel principal (SPA)
|   ├── cambiar-clave.php  Cambio obligatorio de contraseña temporal
|   └── assets/            css/app.css, img/ (logo e ilustraciones SVG), js/,
vendor/ (Bootstrap)
├── src/
|   ├── Core/          Auth, Bitacora, Http, Migraciones
|   ├── Controllers/   16 controladores (reglas de negocio)
|   ├── Models/        15 modelos (acceso a datos)
|   ├── Services/      MicroservicioClient, ReglasClinicas, Correo
|   └── Lib/           PHPMailer 6.9.3
├── tests/             4 baterías de pruebas automatizadas
├── docker-compose.yml        Microservicios en la PC (XAMPP)
└── docker-compose.prod.yml   Sistema completo en Azure
```

Archivos de la interfaz (public/assets/js)

Archivo	Contenido
tema.js	Tema claro u oscuro, cargado antes de pintar la página
app.js	Utilidades (api, avisos, formularios), menú por rol, navegación por #, Inicio, residentes, habitaciones, familiares, evaluaciones, cotizador, usuarios, bitácora
atencion.js	Solicitudes, agenda del especialista, visitas, acompañamientos, configuración
servicios.js	Laboratorio, catálogo de exámenes, farmacia e inventario
caja.js	Resumen de caja, cuentas, pagos, cuotas, fundación, donaciones y gastos
reportes.js	Los 7 reportes, su hoja imprimible y la exportación CSV

Controladores y modelos por módulo

Módulo	Controlador	Modelos
Inicio	DashboardController	(usa varios)
Residentes y alojamiento	ResidenteController, HabitacionController, FamiliarController	Residente, Habitacion, Familiar, PlanServicio
Ficha médica y evaluaciones	FichaController, EvaluacionController	FichaMedica, EvaluacionMedica, Catalogos
Atención médica	SolicitudController, VisitaController	SolicitudMedica, VisitaMedica
Servicios clínicos	LaboratorioController, FarmaciaController	Laboratorio, Farmacia

Módulo	Controlador	Modelos
Finanzas	CajaController, CotizacionController	Caja, CuentaFamiliar
Reportes	ReporteController	Reporte
Administración y cuenta	UsuarioController, ConfiguracionController, CuentaController	Usuario

Convenciones de documentación del código

- Cada archivo inicia con un comentario que dice su ruta y su responsabilidad.
- Los métodos públicos llevan un bloque `/** … */` cuando su regla de negocio no es evidente (por ejemplo, por qué se bloquea una fila o qué devuelve).
- Nombres en español, igual que el dominio: `registrarPago`, `cambiarEstado`, `descuento_fundacion_pct`.
- Las reglas tomadas del análisis citan su origen en el comentario (por ejemplo "según Fase de Diseño, sección 2.4").
- `schema.sql` comenta cada tabla y las columnas con reglas especiales.

3. Flujo de una petición

Toda acción del usuario sigue el mismo camino. Ejemplo: el contador registra un pago de Q 500.

1. **Vista (caja.js).** El formulario se valida en el navegador y `api('pago_registrar', datos)` envía un `POST` JSON a `api.php?action=pago_registrar` con el encabezado `X-CSRF-Token`.
2. **Front Controller (api.php).** Busca la acción en la tabla `$rutas`: `['CajaController', 'registrarPago', [A, C], true]`.
3. **Sesión y rol (Auth::exigir).** Verifica que la sesión exista y siga vigente en la base (usuario activo, misma versión de sesión) y que el rol sea Administrador o Contador. Si no, responde 401 o 403.
4. **CSRF.** Como la acción modifica datos, exige el método `POST` y compara el token; si no coincide, responde 419.

5. **Controlador (CajaController::registrarPago).** Valida cada campo con `Http` (número, fecha, opción) y aplica la regla de negocio: el pago no puede superar el saldo y pide referencia si no es en efectivo.
6. **Transacción (CajaController y modelo Caja).** El controlador abre una transacción y bloquea al residente con `SELECT … FOR UPDATE` para que dos pagos simultáneos no superen el saldo; el modelo inserta el pago con una sentencia preparada y se confirma.
7. **Bitácora.** Registra la acción con usuario, IP y detalle en `bitacora_auditoria`.
8. **Respuesta.** `api.php` devuelve `{"status":"success","data":{…}}`. La vista muestra el aviso y recarga la pantalla.

Si algo falla, el controlador lanza `HttpError` con un mensaje en español y un código HTTP (422 dato inválido, 409 conflicto de negocio, 404 no existe). `api.php` lo convierte en `{"status":"error","message":"…"}`. Los errores de base de datos se traducen a un mensaje comprensible y el detalle técnico va solo al registro de errores de PHP.

4. API del sistema

La interfaz y el servidor se comunican solo por **JSON** a través de `public/api.php?action=<acción>`. Hay **72 acciones: 68** en la tabla `$rutas` y 4 especiales (ingreso, salida, olvido de contraseña y bitácora). Las de método POST modifican datos y exigen el token CSRF.

```
Éxito:  {"status": "success", "data": { ... }}
Error:  {"status": "error", "message": "Texto para el usuario"}   (HTTP 401, 403, 404, 409, 419, 422 o 500)
```

Para agregar una acción basta una línea en `$rutas` (`'accion' => ['Controlador', 'metodo', [roles], modifica]`) y el método en el controlador; api.php aplica la seguridad.

Acción	Método	Controlador::método	Roles
login	POST	Auth::login	Público
olvide_clave	POST	CuentaController::olvideClave	Público
logout	POST	Auth::cerrar	Con sesión
bitacora	GET	Bitacora::listar	Admin
dashboard	GET	DashboardController::resumen	Con sesión

Acción	Método	Controlador::método	Roles
residentes	GET	ResidenteController::listar	Con sesión
residente_expediente	GET	ResidenteController::expediente	Admin, Médico, Enfermero, Especialista
residente_guardar	POST	ResidenteController::guardar	Admin
residente_estado	POST	ResidenteController::cambiarEstado	Admin, Médico
familiares	GET	FamiliarController::listar	Admin, Contador
familiar_guardar	POST	FamiliarController::guardar	Admin
familiar_eliminar	POST	FamiliarController::eliminar	Admin
habitaciones	GET	HabitacionController::listar	Con sesión
habitaciones_disponibles	GET	HabitacionController::disponibles	Admin
habitacion_guardar	POST	HabitacionController::guardar	Admin
habitacion_eliminar	POST	HabitacionController::eliminar	Admin
medicamentos_catalogo	GET	FichaController::catalogoMedicamentos	Admin, Médico, Especialista, Farmacia
padecimiento_guardar	POST	FichaController::guardarPadecimiento	Admin, Médico
padecimiento_eliminar	POST	FichaController::eliminarPadecimiento	Admin, Médico
permanente_guardar	POST	FichaController::guardarPermanente	Admin, Médico
evaluaciones	GET	EvaluacionController::recientes	Admin, Médico
evaluacion_guardar	POST	EvaluacionController::guardar	Admin, Médico
solicitudes_opciones	GET	SolicitudController::opciones	Admin, Médico, Fundación
solicitudes	GET	SolicitudController::listar	Admin, Médico, Fundación, Enfermero, Especialista
solicitud_crear	POST	SolicitudController::crear	Admin, Médico
solicitud_programar	POST	SolicitudController::programar	Admin, Fundación
solicitud_cancelar	POST	SolicitudController::cancelar	Admin, Médico, Fundación

Acción	Método	Controlador::método	Roles
visita_detalle	GET	VisitaController::detalle	Admin, Médico, Especialista
visita_iniciar	POST	VisitaController::iniciar	Admin, Especialista
visita_guardar	POST	VisitaController::guardar	Admin, Especialista
visita_examen_agregar	POST	VisitaController::agregarExamen	Admin, Especialista
visita_examen_quitar	POST	VisitaController::quitarExamen	Admin, Especialista
visita_receta_agregar	POST	VisitaController::agregarReceta	Admin, Especialista
visita_receta_quitar	POST	VisitaController::quitarReceta	Admin, Especialista
visita_finalizar	POST	VisitaController::finalizar	Admin, Especialista
laboratorio	GET	LaboratorioController::listar	Admin, Laboratorio
laboratorio_resultado	POST	LaboratorioController::registrarResultado	Admin, Laboratorio
examenes_catalogo	GET	LaboratorioController::catalogo	Admin, Laboratorio
examen_guardar	POST	LaboratorioController::guardarExamen	Admin, Laboratorio
farmacia	GET	FarmaciaController::listar	Admin, Farmacia
farmacia_entregar	POST	FarmaciaController::entregar	Admin, Farmacia
inventario	GET	FarmaciaController::inventario	Admin, Farmacia
medicamento_guardar	POST	FarmaciaController::guardarMedicamento	Admin, Farmacia
inventario_ingreso	POST	FarmaciaController::ingreso	Admin, Farmacia
caja_resumen	GET	CajaController::resumen	Admin, Contador
cuentas	GET	CajaController::cuentas	Admin, Contador
cuenta_estado	GET	CajaController::estadoCuenta	Admin, Contador
pago_registrar	POST	CajaController::registrarPago	Admin, Contador
cuotas_preview	GET	CajaController::cuotasPreview	Admin, Contador
cuotas_generar	POST	CajaController::generarCuotas	Admin, Contador
fundacion_cuenta	GET	CajaController::deudaFundacion	Admin, Contador, Fundación
fundacion_pagar	POST	CajaController::pagarFundacion	Admin, Contador
donaciones	GET	CajaController::donaciones	Admin, Contador

Acción	Método	Controlador::método	Roles
donacion_registrar	POST	CajaController::registrarDonacion	Admin, Contador
gastos	GET	CajaController::gastos	Admin, Contador
gasto_registrar	POST	CajaController::registrarGasto	Admin, Contador
reportes_opciones	GET	ReporteController::opciones	Admin, Contador, Médico, Fundación
reporte	GET	ReporteController::generar	Admin, Contador, Médico, Fundación
cotizador_catalogo	GET	CotizacionController::catalogo	Admin, Contador
cotizar	POST	CotizacionController::cotizar	Admin, Contador
usuarios	GET	UsuarioController::listar	Admin
usuario_guardar	POST	UsuarioController::guardar	Admin
usuario_restablecer	POST	UsuarioController::restablecer	Admin
solicitud_clave_descartar	POST	UsuarioController::descartarSolicitudClave	Admin
cambiar_clave	POST	CuentaController::cambiarClave	Con sesión
configuracion	GET	ConfiguracionController::obtener	Admin
configuracion_guardar	POST	ConfiguracionController::guardar	Admin
correo_probar	POST	ConfiguracionController::probarCorreo	Admin
correo_diagnostico	POST	ConfiguracionController::diagnosticarCorreo	Admin

Acciones agregadas en la versión v1.4 (29 oct 2026): **caja_anular** (POST, CajaController::anular, Admin y Contador) anula con motivo un pago, pago a la fundación, donación o gasto; **laboratorio_corregir** (POST, LaboratorioController::corregirResultado, Admin y Laboratorio) corrige un resultado con motivo y conserva el original; sesion_ping (GET, CuentaController::ping, con sesión) renueva la sesión desde el aviso «Seguir conectado».

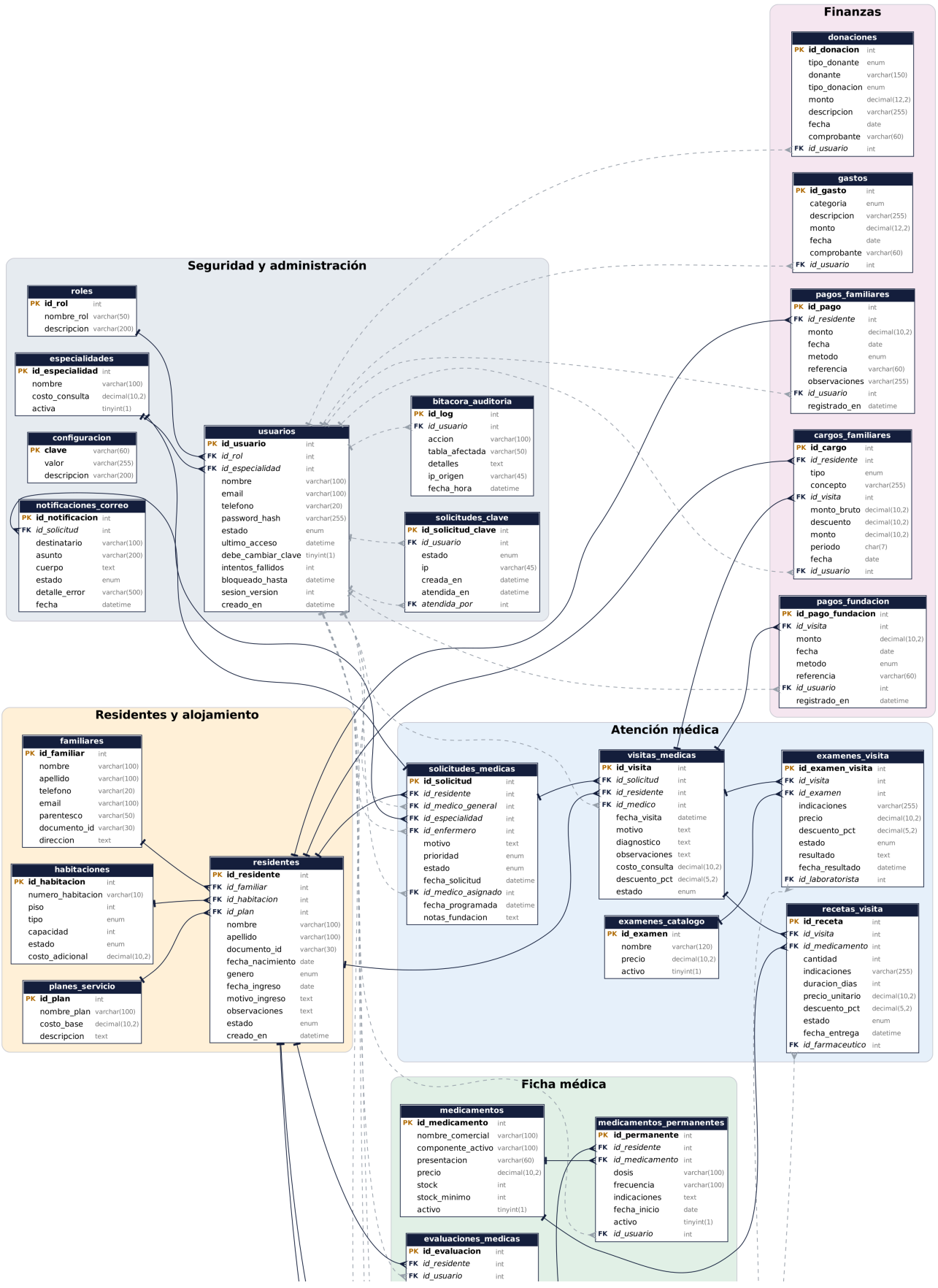
Cambios de la versión v1.5 (30 oct 2026), sin acciones nuevas: los correos al personal (contraseña temporal, solicitud de contraseña y correo de prueba) incluyen el botón **Ingresar al sistema**, con la dirección que arma `Http::urlSistema()` a partir del servidor que atendió la petición, así que sirve igual en XAMPP y en Azure.

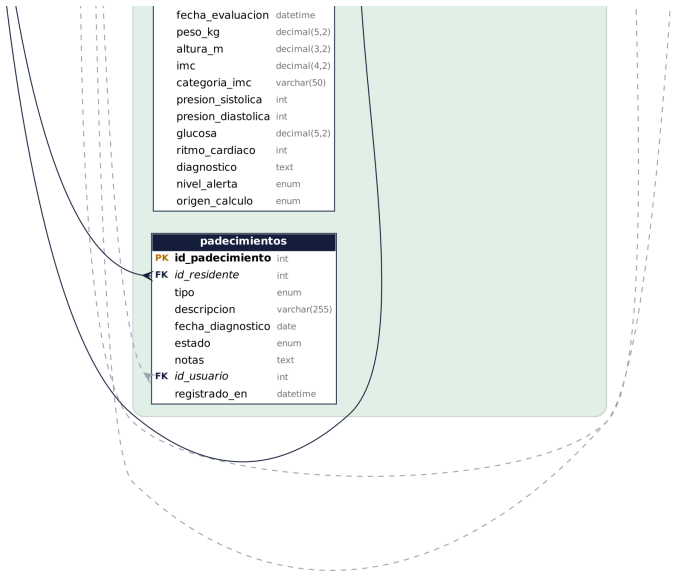
5. Base de datos

La base `asilo_cabeza_algodon` usa el motor **InnoDB** con juego de caracteres **utf8mb4**: 25 tablas, 39 llaves foráneas y datos de ejemplo. El script completo es `database/schema.sql`. El diagrama se generó desde ese script, por lo que coincide con la base instalada.

Modelo entidad-relación — Asilo Cabeza de Algodón

25 tablas · MariaDB/MySQL · base asilo_cabeza_algodon · PK = llave primaria, FK = llave foránea





Integridad de los datos

- **Llaves foráneas** con reglas explícitas: `RESTRICT` donde borrar perdería historia (un residente con cargos), `CASCADE` en datos dependientes (la ficha médica de un residente) y `SET NULL` en referencias de auditoría (quién registró).
- **Valores únicos:** correo de usuario, DPI de familiar y residente, número de habitación, una visita por solicitud y una cuota por residente y mes (`uq_cuota_periodo`).
- **Transacciones:** 12 operaciones que tocan varias tablas (finalizar visita, entregar receta, registrar pago, generar cuotas, cambiar estado de residente, entre otras) usan `beginTransaction` / `commit` / `rollBack` .
- **Concurrencia:** 8 bloqueos `SELECT … FOR UPDATE` evitan que dos usuarios entreguen la misma receta, dejen existencias negativas o paguen más que el saldo al mismo tiempo.
- **Sentencias preparadas** en todas las consultas con datos del usuario (PDO), lo que impide la inyección SQL.

Conexión. `config/database.php` implementa el patrón **Singleton**: una sola conexión PDO por petición, con errores como excepciones, `utf8mb4` y sentencias preparadas reales (sin emulación). Al compartir la conexión, una transacción abarca a todos los modelos que intervienen.

Migraciones automáticas. `src/Core/Migraciones.php` compara la versión de la base con la del código una vez por sesión. Si faltan columnas o tablas nuevas, las agrega con `ALTER TABLE` / `CREATE TABLE` sin borrar datos. Así, actualizar el sistema en XAMPP o en Azure no exige volver a importar el script. Cada paso es idempotente: se puede ejecutar varias veces sin efecto adicional.

Respaldo. En Azure es automático: `instalar.sh` programa `deploy/azure/respaldo.sh` todos los días a las 23:30 (hora de Guatemala); guarda la base comprimida en `/opt/asilo-respaldos` y conserva los últimos 14. Respaldo manual: `bash deploy/azure/respaldo.sh`. Restaurar: `bash deploy/azure/respaldo.sh restaurar /opt/asilo-respaldos/<archivo>.sql.gz`. En XAMPP: phpMyAdmin > Exportar.

6. Microservicios

Los dos microservicios son aplicaciones **Flask** independientes que corren con **gunicorn** (2 procesos) en su propio contenedor Docker (`python:3.10-slim`). Reciben y devuelven JSON, validan sus entradas y no tienen acceso a la base de datos.

Microservicio	Puerto	Rutas	Entrada	Salida
ms-imc	5001	`GET /health` · `POST /api/v1/imc` · `POST /calcular-imc`	`peso_kg` (20–250), `altura_m` (1.0–2.3)	`imc`, `categoria`, `nivel`, `alerta_nutricional`
ms-cotizador	5002	`GET /health` · `POST /api/v1/cotizar` · `POST /cotizar`	`costo_base`, `costo_habitacion`, `nivel_cuidados` (bajo, medio, alto), `servicios_extra[]`, `descuento_pct` (0–50)	`subtotal`, `recargo_cuidado`, `monto_descuento`, `monto_total`

Reglas que aplican

- **IMC geriátrico:** IMC = peso / estatura². Menor de 22 = bajo peso (Atención); 22 a 27 = normal; más de 27 a 30 = sobrepeso; más de 30 = obesidad (Atención). El controlador combina este nivel con la presión y la glucosa para decidir si la alerta es Urgente.
- **Cotización:** subtotal = plan + habitación + recargo por nivel de cuidado (Q 0, Q 100 o Q 250) + servicios extra; el descuento se resta del subtotal.

Cliente y respaldo. `src/Services/MicroservicioClient.php` llama a los microservicios con cURL y un límite de 3 segundos (Gateway). Si no responden, `src/Services/ReglasClinicas.php` aplica las mismas fórmulas en PHP y el resultado se guarda con `origen_calculo` = local (microservicio cuando sí responde), de modo que el usuario nunca queda bloqueado. El Inicio muestra si cada microservicio está en línea (`/health`).

Prueba manual (en la PC con Docker):

```
curl -X POST http://localhost:5001/api/v1/imc -H "Content-Type:
application/json" -d "{\"peso_kg\": 65, \"altura_m\": 1.62}"
```

7. Seguridad

La seguridad se aplica en el servidor, en un solo punto (`api.php` + `Auth`); ocultar un botón en la interfaz es solo comodidad, nunca la protección.

Control	Cómo está implementado	Dónde
Contraseñas	Cifrado Bcrypt (`password_hash`); mínimo 8 caracteres con letras y números	Auth, CuentaController, Usuario
Sesión	Cookie `HttpOnly` , `SameSite=Lax` , `Secure` con HTTPS; nuevo id al ingresar; cierre tras 2 horas sin actividad	Auth::iniciarSesion
Control de acceso por rol (RBAC)	Cada ruta declara sus roles; 8 roles; respuesta 403 si no corresponde	api.php `$rutas` , Auth::exigir
Sesión vigente	En cada petición se verifica que el usuario siga activo y que su `sesion_version` no haya cambiado (desactivado, rol cambiado o contraseña restablecida)	Auth::vigente
CSRF	Token aleatorio de 64 caracteres por sesión, enviado en `X-CSRF-Token` y comparado con `hash_equals` en toda acción que modifica datos	Auth::verificarCsrf
Fuerza bruta	5 intentos fallidos seguidos bloquean la cuenta 15 minutos; aviso desde el 3.er intento	Auth::login

Control	Cómo está implementado	Dónde
Recuperación de acceso	Solicitud sin revelar si el correo existe, límite de 3 por 10 minutos; contraseña temporal con cambio obligatorio	CuentaController, UsuarioController
Inyección SQL	Sentencias preparadas PDO sin emulación	Todos los modelos
XSS	Todo texto se escapa al pintarlo (`esc()` en JS, `htmlspecialchars` en PHP)	app.js, vistas PHP
Validación	Tipo, rango, longitud, fechas y opciones en el servidor con mensajes claros; DPI validado como CUI (13 dígitos, departamento, municipio y dígito verificador), teléfonos de 8 dígitos y nombres solo con letras	Core/Http
Auditoría	Bitácora de cada acción: usuario, fecha, tabla, detalle e IP	Core/Bitacora
Transporte	HTTPS con certificado de Let's Encrypt; HTTP e IP redirigen a HTTPS	Caddy
Exposición mínima	Solo `public/` es raíz web; base y microservicios solo en la red interna de Docker; encabezados `nosniff`, `SAMEORIGIN`, `Referrer-Policy`; PHP sin mostrar errores	Dockerfile, docker-compose.prod.yml
Secretos	Contraseñas de la base aleatorias en `.env` del servidor; `correo.php` y `.env` fuera de GitHub (`.gitignore`)	instalar.sh

8. Configuración, despliegue y mantenimiento

Los mismos archivos funcionan en XAMPP y en Docker: `config/config.php` toma primero las **variables de entorno** y, si no existen, los valores de XAMPP.

Variable	Valor en XAMPP	Valor en Azure (Docker)
DB_HOST / DB_PORT	localhost / 3306	db / 3306
DB_NAME	asilo_cabeza_algodon	asilo_cabeza_algodon
DB_USER / DB_PASS	root / (vacía)	asilo / aleatoria en `.env`
MS_IMC_URL	http://localhost:5001	http://ms-imc:5001
MS_COTIZADOR_URL	http://localhost:5002	http://ms-cotizador:5002
CORREO_ACTIVO, CORREO_METODO, CORREO_APPS_SCRIPT_URL, CORREO_APPS_SCRIPT_CLAVE	`config/correo.php`	`.env` (con `deploy/azure/correo.sh`)
DOMINIO	—	asilo-cabeza-algodon.canadacentral.cloudapp.azure.com

Otros parámetros fijos en `config.php`: nombre del asilo, zona horaria `America/Guatemala`, tiempo máximo de los microservicios (3 s) e identificadores de los 8 roles.

Diferencias entre XAMPP y Azure que el código resuelve (revisadas en la v1.5):

- **IP del usuario:** en Azure todas las peticiones llegan por el proxy HTTPS, así que `Http::ip()` toma la IP real del encabezado `X-Forwarded-For` (solo cuando quien conecta es una dirección interna). La bitácora guarda esa IP.
- **Hora:** `config/database.php` ejecuta `SET time_zone = '-06:00'` al conectar, para que `NOW()` de la base de datos y `date()` de PHP den la misma hora de Guatemala aunque el servidor esté en UTC.
- **Duración de la sesión:** PHP borra por defecto las sesiones con 24 minutos sin uso; `Auth::iniciarSesion()` sube `session.gc_maxlifetime` a 2 horas y 10 minutos para respetar el cierre por inactividad de 2 horas.
- **Correo:** en XAMPP se lee `config/correo.php`; en Azure, el `.env` del servidor (`deploy/azure/correo.sh`). Ninguno de los dos se sube a GitHub.

Contenedores en Azure (`docker-compose.prod.yml`)

Contenedor	Imagen	Expuesto
asilo_proxy	caddy:2-alpine	Puertos 80 y 443
asilo_web	php:8.2-apache + pdo_mysql (`docker/web/Dockerfile`)	Solo red interna
asilo_db	mariadb:10.11, volumen `datos_asilo`	Solo red interna
asilo_ms_imc / asilo_ms_cotizador	python:3.10-slim + gunicorn	Solo red interna

Tareas de mantenimiento

Tarea	Cómo
Actualizar el sistema	Push a GitHub y en Azure ejecutar `instalar.sh` (Ejecutar comando)
Ver el estado	`cd /opt/asilo-cabeza-algodon && docker compose -f docker-compose.prod.yml ps`
Ver registros de errores	`docker compose -f docker-compose.prod.yml logs --tail=50 web proxy db`
Consultar la base	`docker compose -f docker-compose.prod.yml exec -T db sh -c 'mariadb -uasilo -p"$MARIADB_PASSWORD" asilo_cabeza_algodon -e "SHOW TABLES;"'`
Activar correos	`bash deploy/azure/correo.sh "URL de Apps Script" "clave"`
Volver a los datos de ejemplo	`bash deploy/azure/reiniciar-datos.sh` (borra los datos; antes guarda un respaldo y conserva el certificado HTTPS y el correo)
Respaldar la base	`mariadb-dump` automático diario a las 23:30 con deploy/azure/respaldo.sh (sección 5); en XAMPP, phpMyAdmin > Exportar
Renovar el certificado HTTPS	Automático (Caddy, cada 60–90 días)

Tarea	Cómo
Ahorrar crédito	Detener la máquina virtual; al encenderla los contenedores arrancan solos (`restart: unless-stopped`)

9. Pruebas automatizadas

El proyecto incluye **5 baterías con 216 casos**, y todas pasan al 100 % en la versión v1.4. Se ejecutan con el sistema encendido y la base recién importada; cada una deja su evidencia en `tests/` .

Batería	Archivo	Casos	Qué prueba
Funcionales por requerimiento	`tests/ejecutar_pruebas.php`	92	Cada caso cita su requerimiento (RF-01 a RF-47) y recorre la API como cada rol
API	`tests/prueba_api.py`	41	Autenticación, permisos por rol, validaciones, CSRF y reglas de negocio
Interfaz	`tests/prueba_ui.py`	14	Navegador real (Playwright): ingreso, menú por rol, formularios, vista móvil de 390 px
Casos límite de cuentas	`tests/prueba_cuentas.py`	37	¿Olvidó su contraseña?, temporal, bloqueo, cierre de sesión remoto

Quinta batería, `tests/prueba_casos_limite.py` (32 casos, CL-01 a CL-32): DPI con letras o mal formado, teléfonos y nombres inválidos, cuota proporcional al ingresar a mitad de mes, anulación de pagos, donaciones y gastos, y corrección de resultados de laboratorio. Se ejecuta igual que las demás: `python tests/prueba_casos_limite.py http://localhost/asilo-cabeza-algodon/public` .

Cómo ejecutarlas (Windows, XAMPP)

```
C:\xampp\php\php.exe tests\ejecutar_pruebas.php http://localhost/asilo-
cabeza-algodon/public
pip install requests playwright
playwright install chromium
python tests/prueba_api.py      http://localhost/asilo-cabeza-algodon/public
python tests/prueba_ui.py       http://localhost/asilo-cabeza-algodon/public
python tests/prueba_cuentas.py  http://localhost/asilo-cabeza-algodon/public
```

Las pruebas crean registros; al terminar se vuelve a importar `database/schema.sql`. El plan de pruebas con fechas, responsables y resultados está en el documento **Plan de Pruebas**.